

11/June/2025

Chapter-1

Flowcharts and Pseudocodes

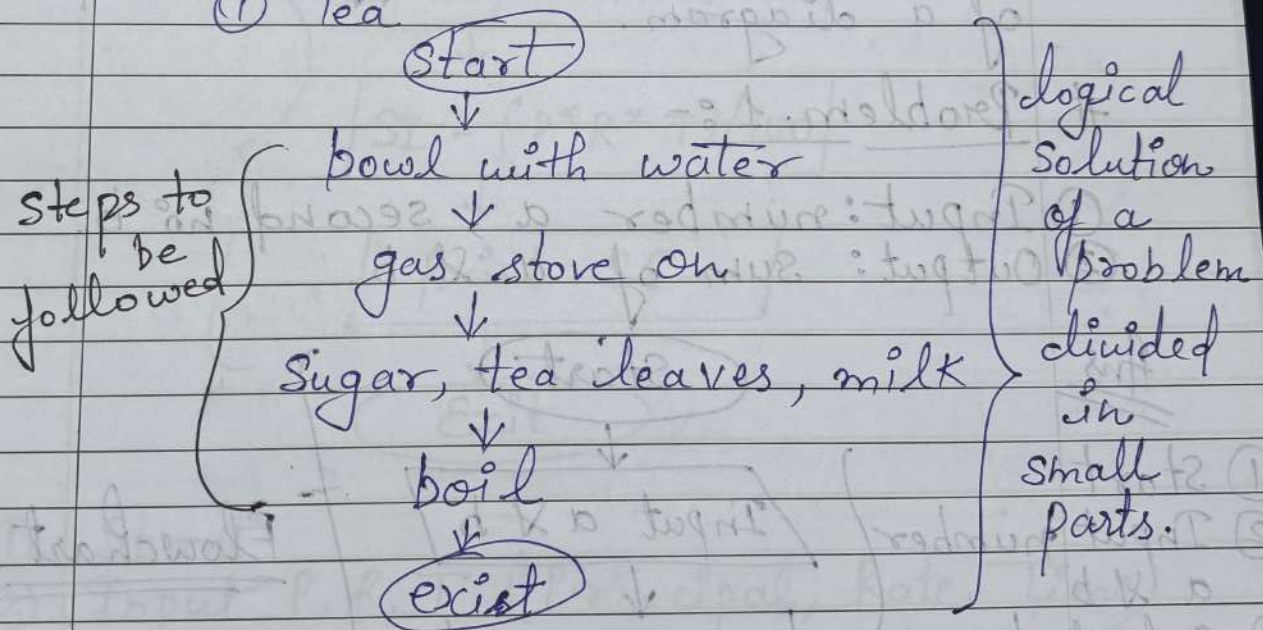
- Flowcharts:- Diagram to represent solutions of problems.

↓
① Small parts

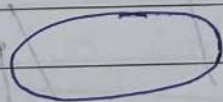
↓
② logically arrange

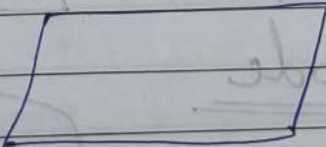
Eg:- To make tea:-

① Tea



Components of Flowchart:-

① Start/exit → 

② Input/output → 

③ Process $\rightarrow$ $a = 10;$

④ Decision $\rightarrow$ $\begin{array}{c} \text{is} \\ a > 10 \\ \text{No} \end{array}$ Yes

⑤ Arrows $\rightarrow$ $\downarrow$

- Two branches emerge from the decision block ① Yes and ② No.
- Arrow connects the different shapes of a diagram.

Problem. 1 :-

① Input: number a , second no b .

② Output: Sum of a & b .

Ans

① Start

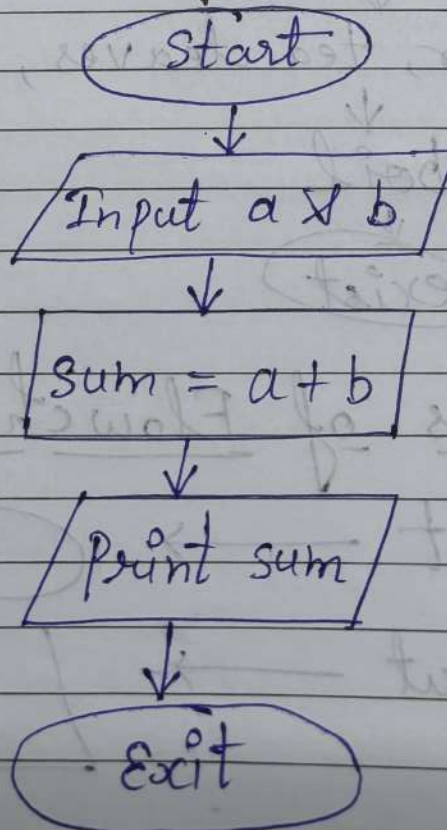
② Input number a & b

③ Calculate
 $Sum = a + b$

④ Print sum

⑤ Exit

pseudocode



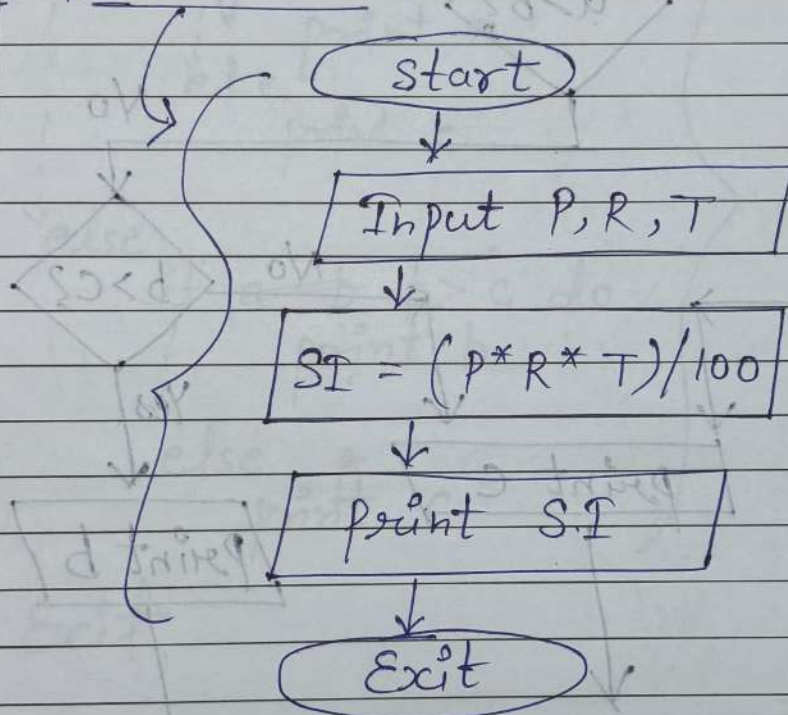
Flowchart

//_

Calculate Simple Interest :-

- Input: Principal, P
Rate, R
Time, T
- Output: $SI = PRT/100$

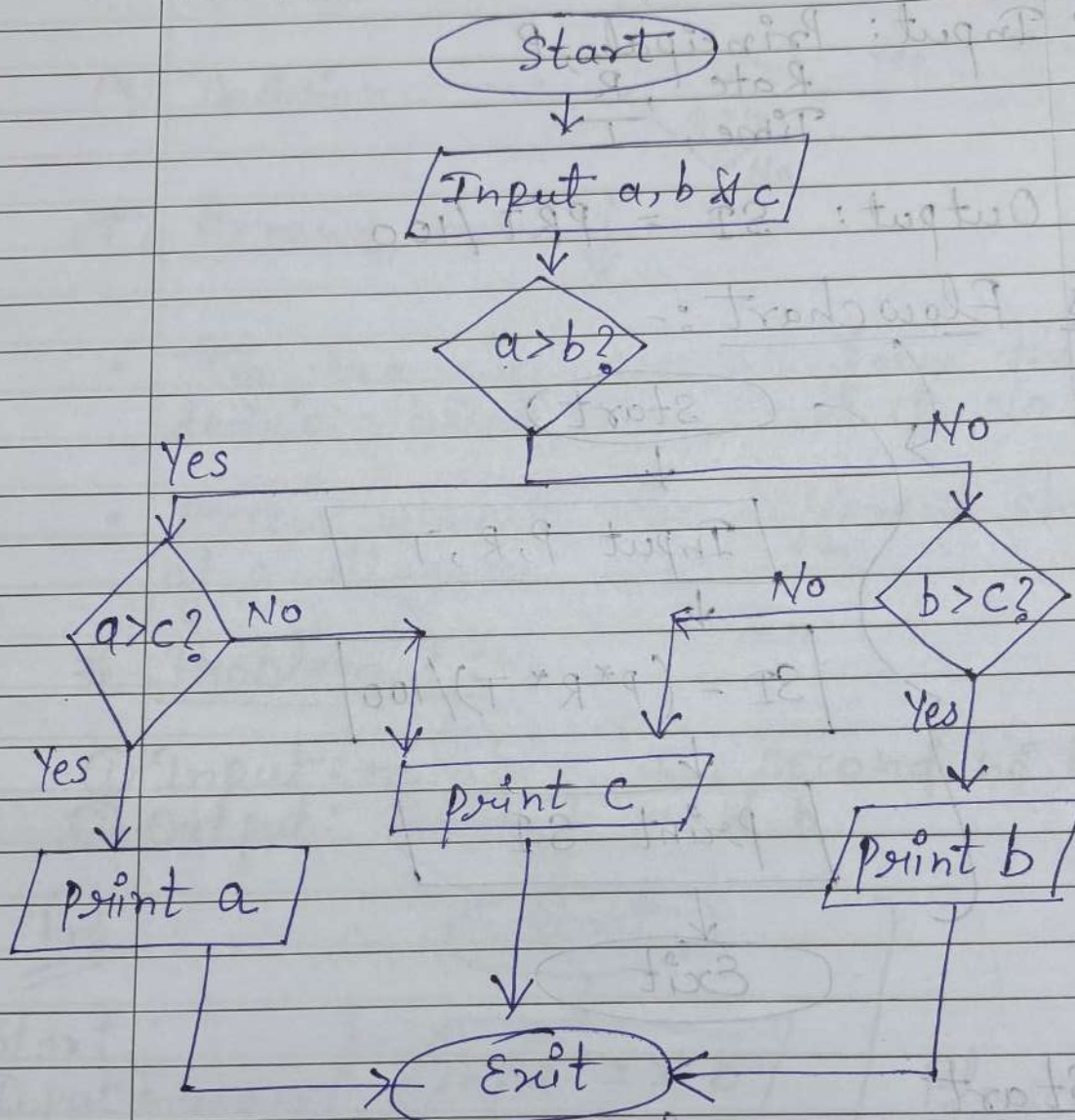
Ans Flowchart :-



- ① Start
- ② Input P, R, T (Principal, Rate, Time)
- ③ Calculate $SI = (P * R * T) / 100$
- ④ Print SI
- ⑤ Exit

↓
pseudocode

Find maximum of 3 Numbers :-



Pseudocode :-

- ① Start
- ② Input a, b & c
- ③ if $a > b$ do
 if $a > c$ do
 print a

else
print c

OR (Full Solution)

① Start
② Input a, b & c

③ If $a > b$ do
 if $a > c$ do
 print a
 else
 print c

else
 if ~~a < b~~ $b > c$ do
 print b
 else
 print c

pseudocode
for max
of 3
numbers.

④ Exit.

⇒ Initially:-

$$\begin{aligned} \text{Sum} &= 0 \\ \text{Sum} &= \text{Sum} + \text{val} \\ &= 0 + 1 = 1 \\ \text{Sum} &= 1 + 2 = 3 \\ \text{Sum} &= 3 + 3 = 6 \\ \text{Sum} &= 6 + 4 = 10 \end{aligned}$$

$n = 4$ 1 1 1 1

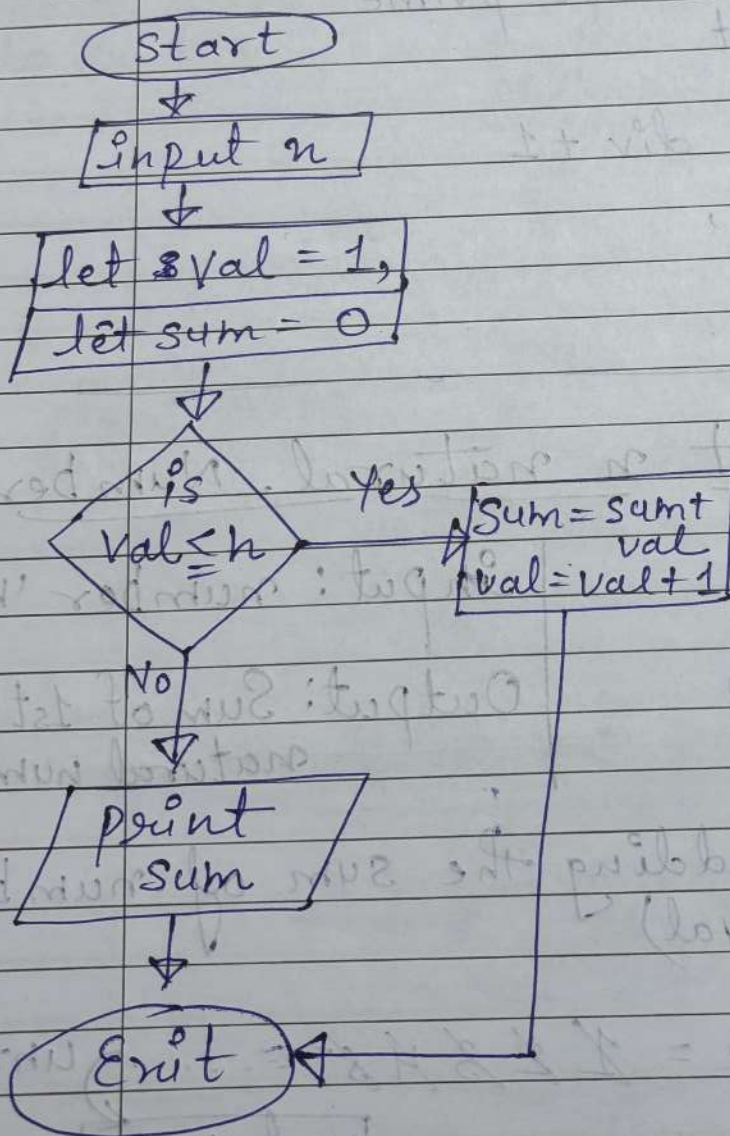
$\text{val} = \cancel{1} \cancel{2} \cancel{3} \cancel{4} 5$

$\downarrow$

$n = 4$

$\text{Sum} = 10$

Flowchart



Pseudocode

- ① Start
- ② Input n
- ③ let $\text{val} = 1$ and $\text{sum} = 0$
- ④ while $\text{val} < n$ do
 $\text{sum} = \text{sum} + \text{val}$
 $\text{val} = \text{val} + 1$
- ⑤ Print sum
- ⑥ Exit

pseudocode :- (for checking prime) :-

- ```

① Start
② Input n
③ let div = 2
④ while div < n do
 if n % div == 0 do
 print "Not prime"
 Exit
 else
 div = div + 1
⑤ Print 'prime'
⑥ Exit

```

### # Sum of first n natural Numbers:-

- |                          |                                        |
|--------------------------|----------------------------------------|
| ① <del>Print</del> Start |                                        |
| ② Input $n$              | input: number ' $n$ '                  |
| ③ let $sum = 0$          | Output: Sum of 1st $n$ natural numbers |
| ④                        |                                        |

⌘ Here we are adding the sum of numbers from 1 to  $n(\text{val})$

eg:- variable value = 1 2 3 4 5 ... until  
 $\boxed{val \leq n}$   
 $\rightarrow \text{Sum} = \text{Sum} + \text{value}$



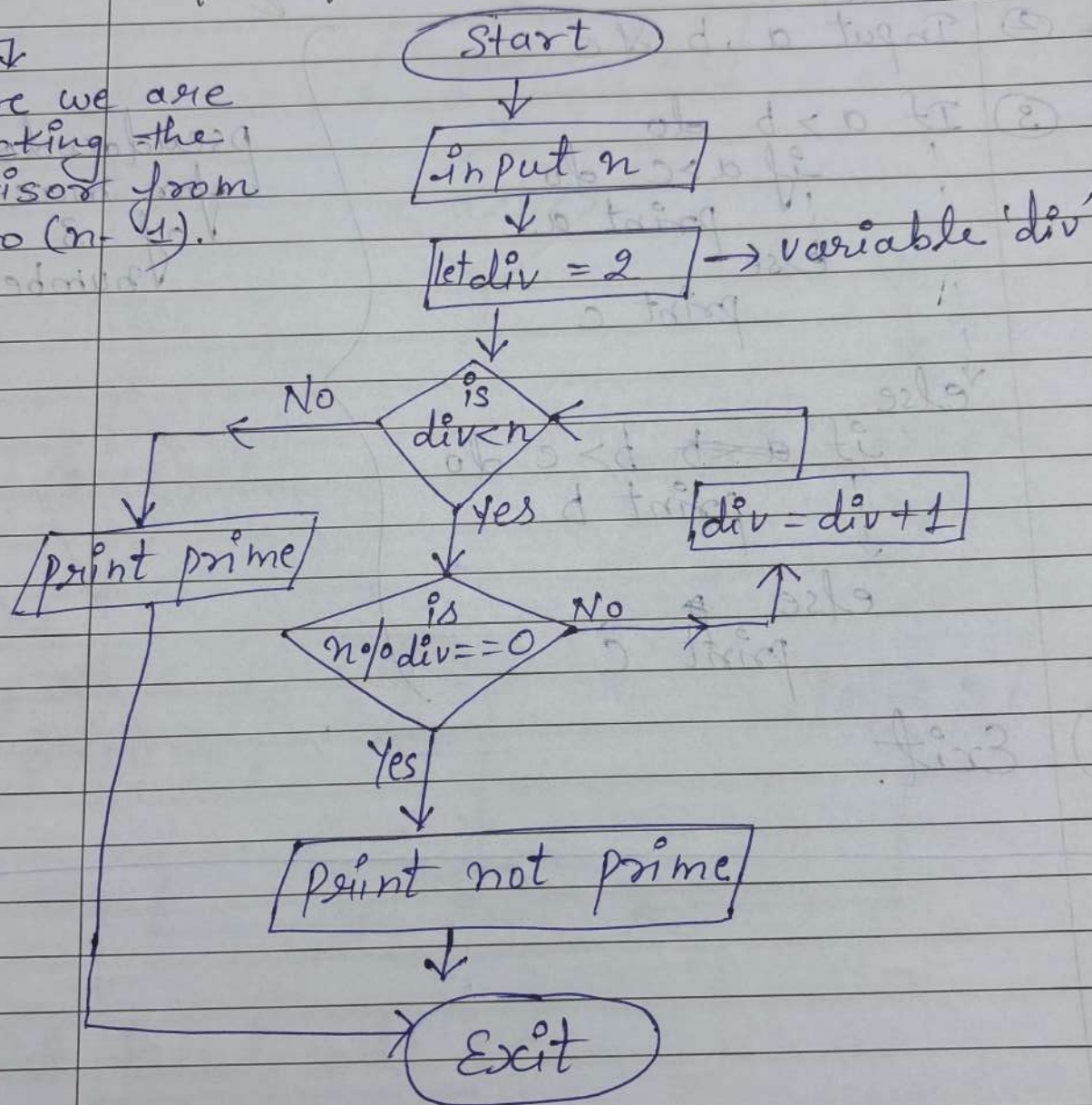
# Find if Number is prime :-

Input: number,  $n$

Output: prime or not prime

⑦ ↓

Here we are checking the divisor from 2 to  $(n-1)$ .



Flowchart



# Print the even numbers b/w 9-100:

Ans Pseudocode :-

(\*) Mistake - ①

① Start

② let  $n = 9$

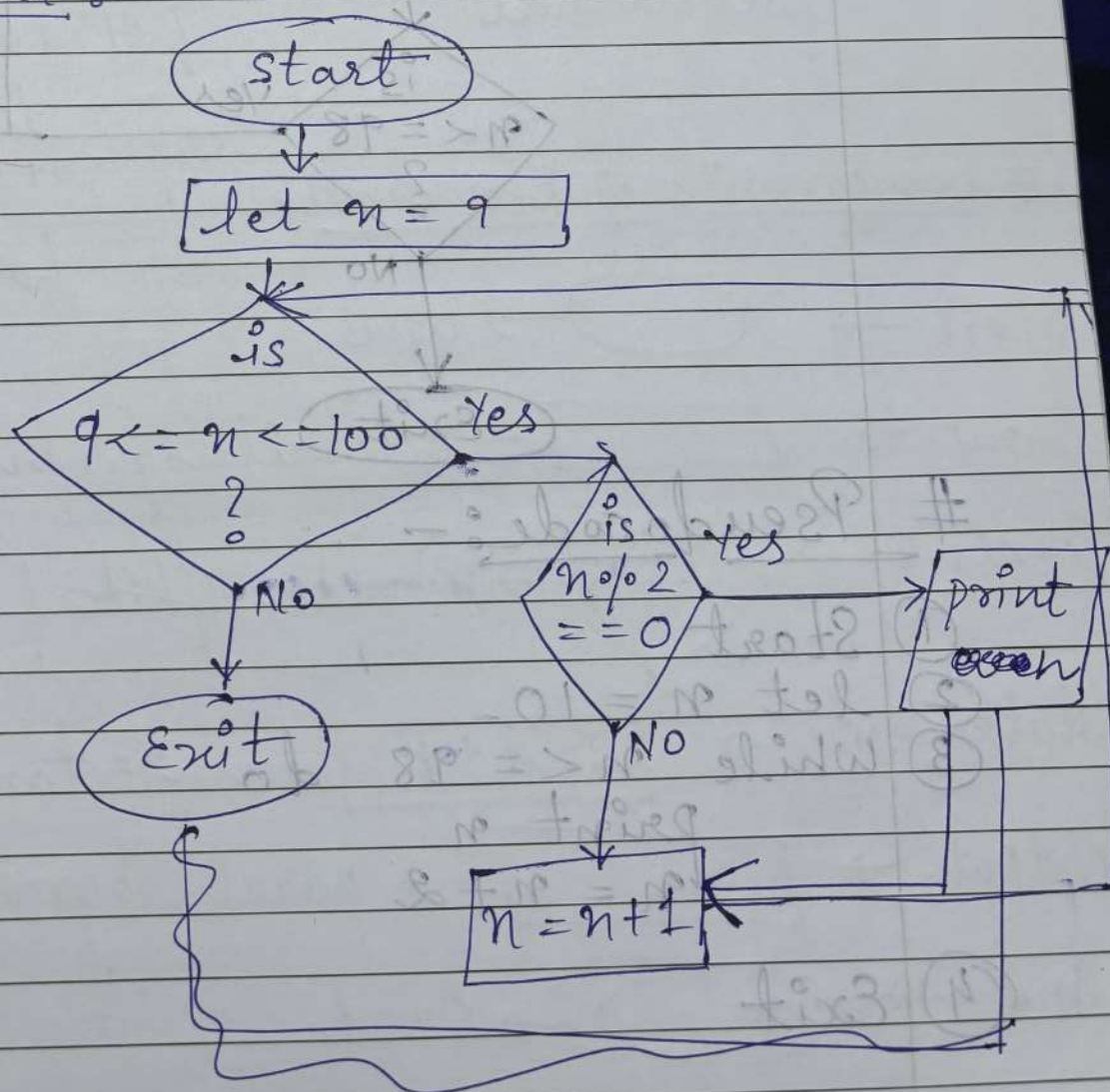
③ while  $9 \leq n \leq 100$  do  
if  $n \% 2 == 0$  do  
print ~~even~~  $n$

$n = n + 1$

④ Exit

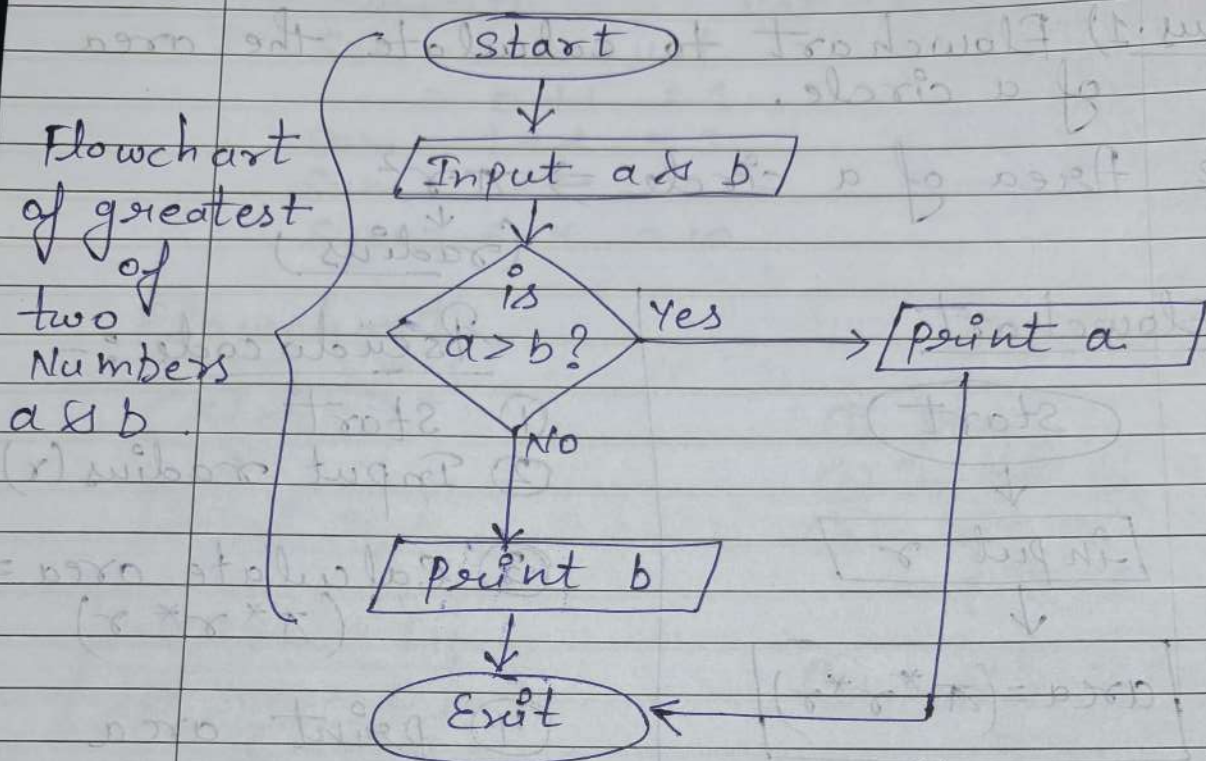
↓  
between  
means  
not to  
include  
those  
Numbers

Flowchart :-





⇒ Flowchart :-



Ques. 3) Flowchart to print even numbers between 9 and 100.

Ans - ① Start

② Input number 'n'

③ Check:

If  $9 \leq n \leq 100$  do  
if  $n \% 2 == 0$   
print ~~an~~ even

else

print odd

else

print 'invalid no'

④ Exit

This is to check if a given no that lies between 9 - 100 is even or odd.

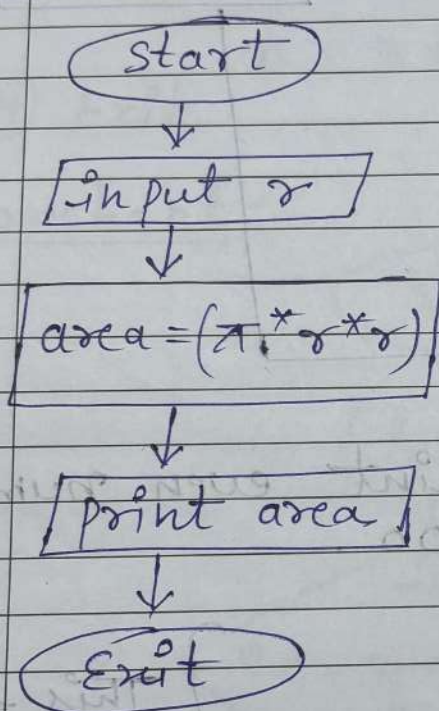


## # Assignment Questions (Flowchart) :-

Ques 1) Flowchart to calculate the area of a circle.

Ans Area of a circle  $\Rightarrow \pi r^2$   
(radius)

Flowchart



Pseudo code :-

- ① Start
- ② Input radius (r)
- ③ Calculate area =  $(\pi * r * r)$
- ④ print area
- ⑤ Exit

Ques 2) Flowchart to find greatest of 2 numbers ?

Ans-2) Pseudo code :-

- ① Start
- ② Input a & b
- ③ Check:  
if  $a > b$   
    print a  
else  
    print b
- ④ Exit



\_/\_/\_

\* Algorithm is a logical approach to solve any problem.

### # Sum of two Numbers:-

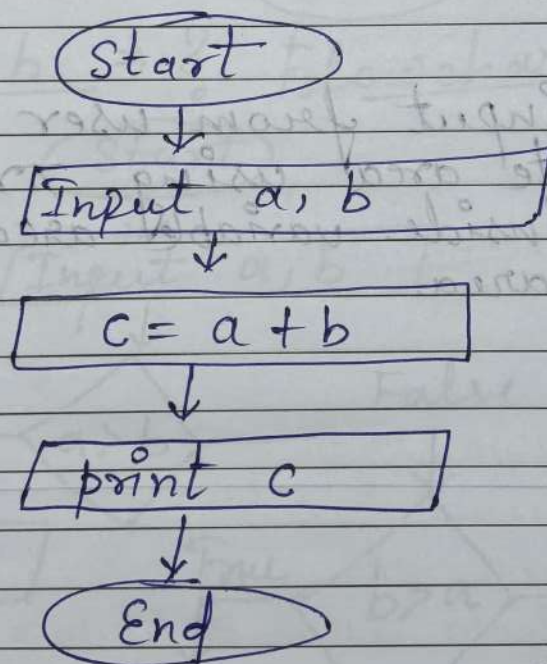
① → Take input from user a and b from user.

→ ② add 'a' and 'b' and store it inside 'c'.

→ ③ print variable 'c'.

→ ④ End.

Flowchart :-



Flowchart to take the sum of 2 Numbers.



19/July/25 # Practice Session: 1.  
live Session { Flowcharts & pseudocode }

- Flowchart :- Diagram to represent solutions of a problem.

Ques: 1) Start a car

→ open door  
→ sit inside car  
→ start car

↓  
pseudocode

Flowchart

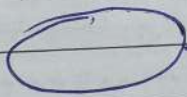
↓  
Start

↓  
[open]

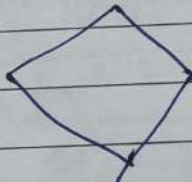
↓  
[sit inside]

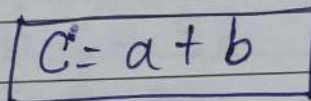
↓  
[car start]  
↓  
End

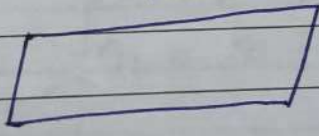
# Components of Flowchart:-

Start →  → oval

Input →  → parallelogram

Condition →  → Rhombus / kite / diamond

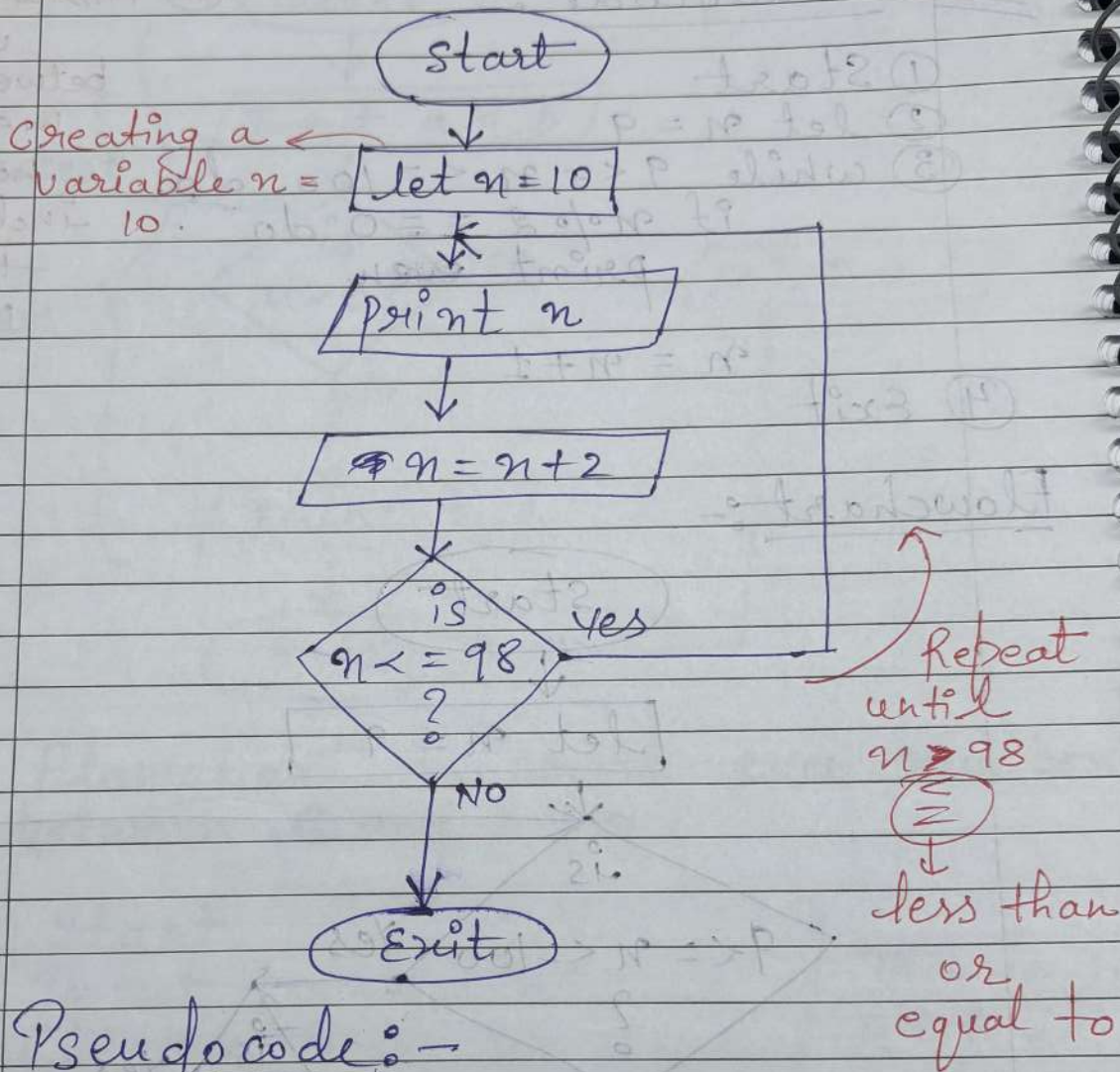
statements →  → rectangle

Output →  → parallelogram

End →  → Oval



# Flowchart to print even numbers between 9 and 100.



# Pseudocode :-

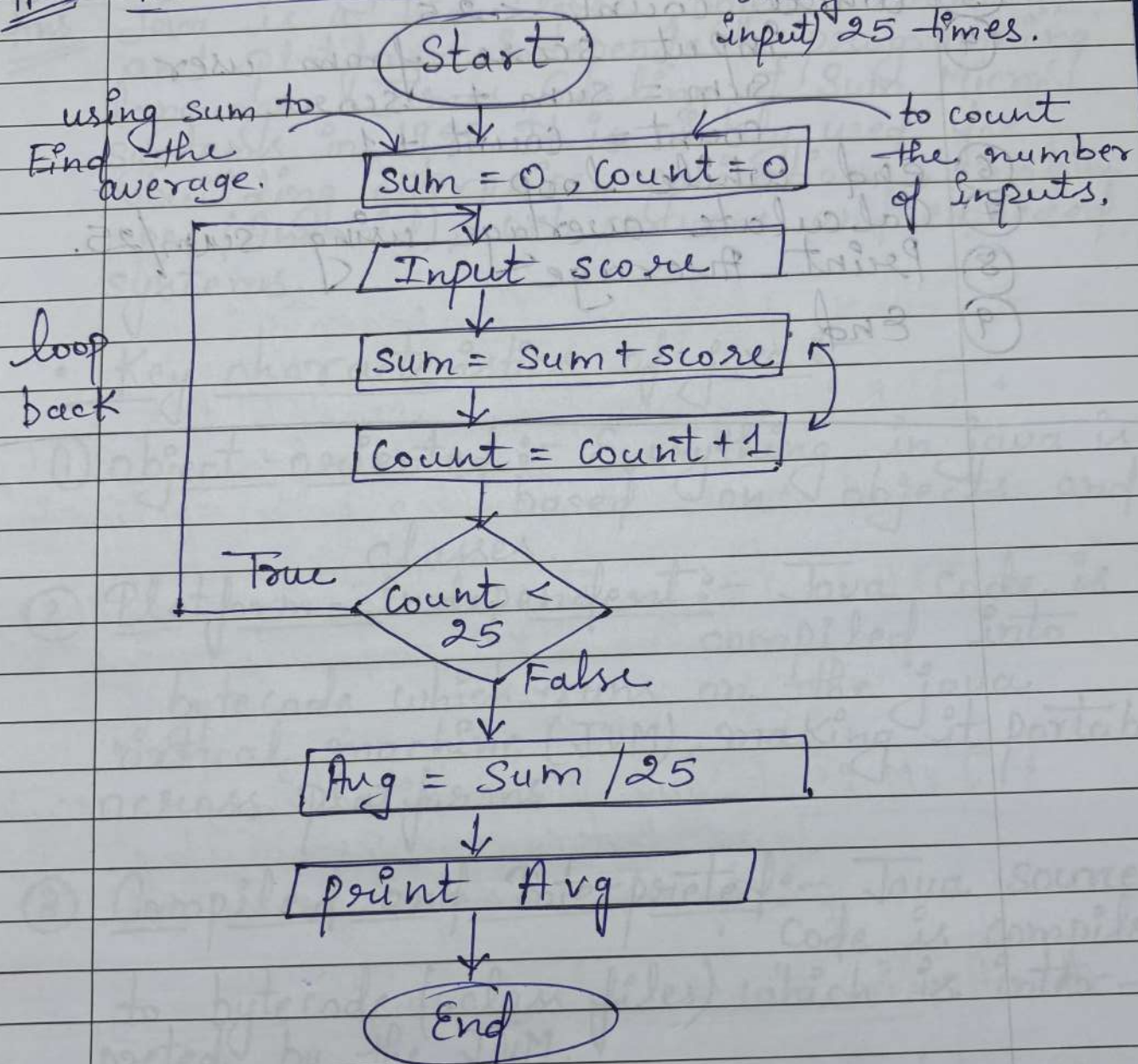
- ① Start
- ② let  $n = 10$
- ③ While  $n \leq 98$  do  
    print  $n$   
     $n = n + 2$
- ④ Exit



Ques. 2) Flowchart for calculating the average from 25 exam scores.

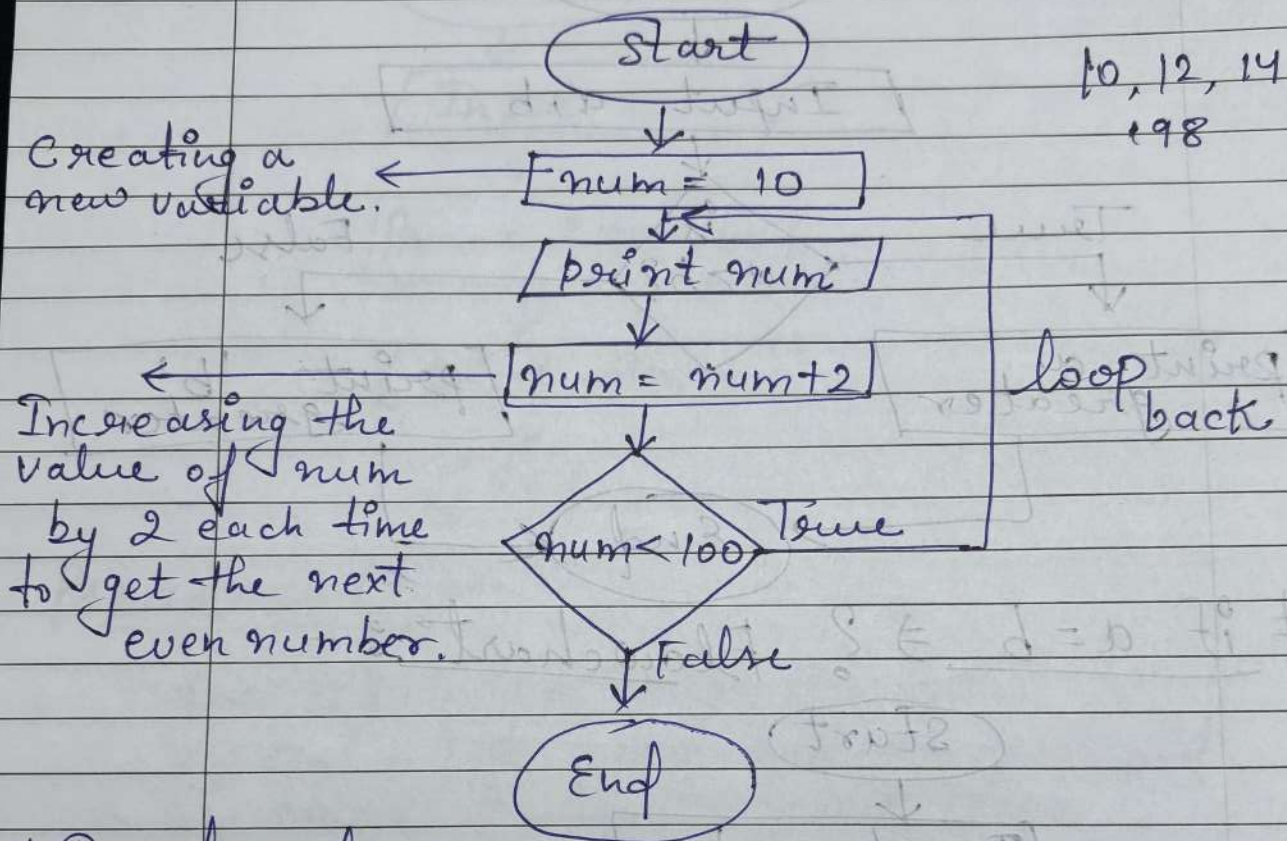
Ans- Flowchart :-

# Here we are taking marks (as input) 25 times.





Ques. 1) Flowchart to print all the even numbers between 9-100?  
 ↳ Here 9 and 100 are not inclusive.

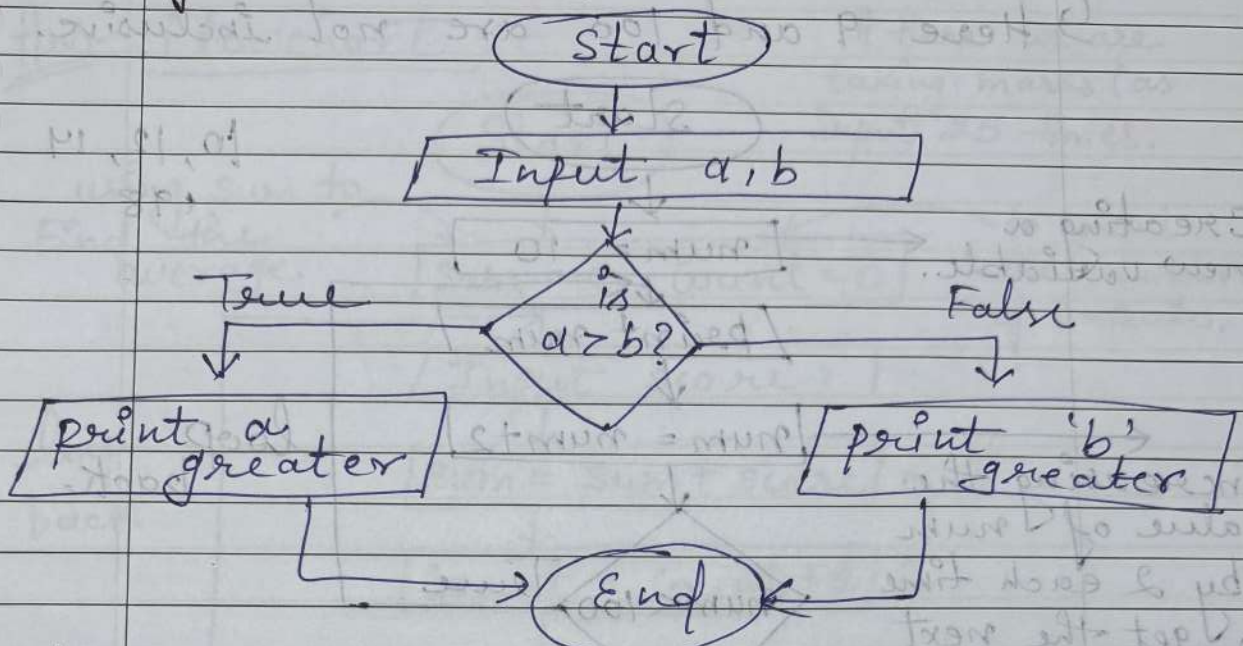


# Pseudocode :-

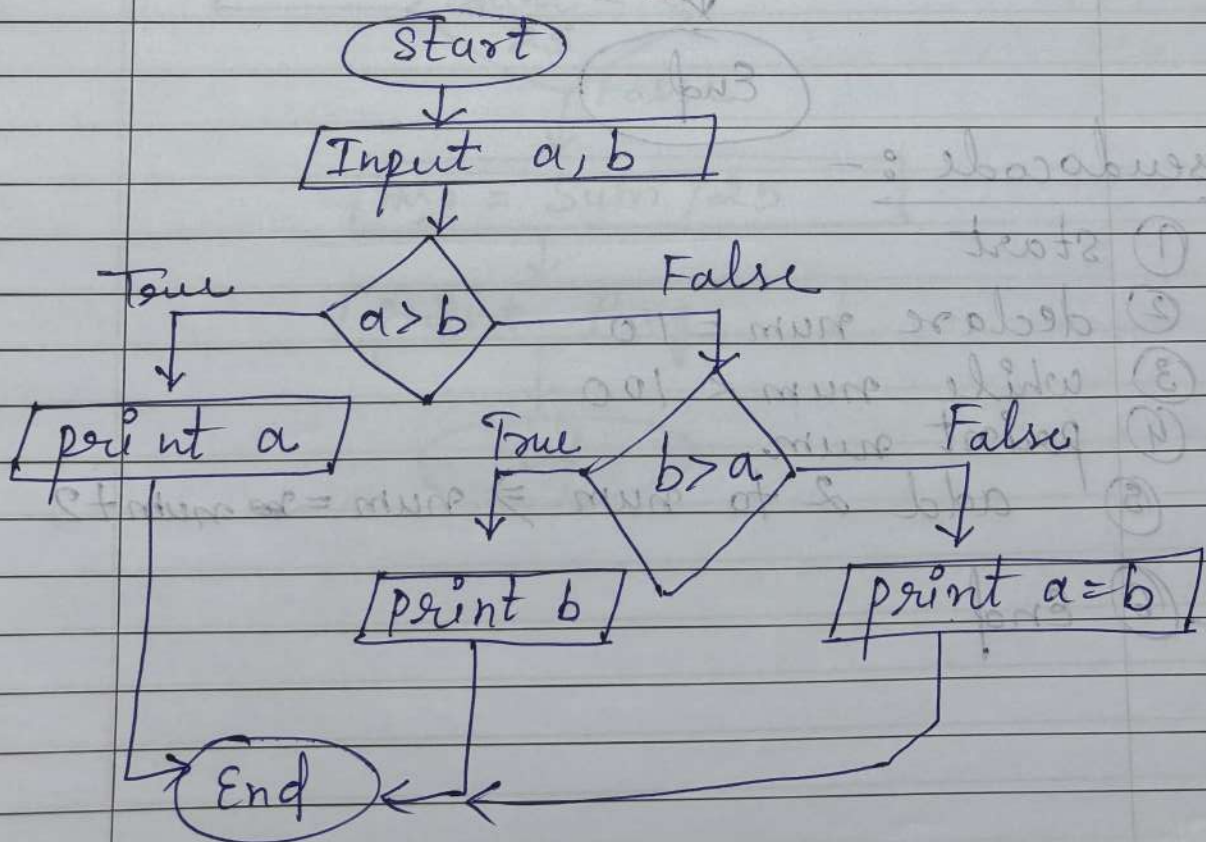
- ① start
- ② declare  $\text{num} = 10$
- ③ while  $\text{num} < 100$
- ④ print num
- ⑤ add 2 to num  $\Rightarrow \text{num} = \text{num} + 2$
- ⑥ End



# Flowchart to find greatest of two Numbers:-



# if  $a = b$  ? Flowchart :-





## # Java Program example:-

Boiler-plate  
Code  
in  
Java

```
public class HelloWorld {
 public static void main(String args[]) {
 System.out.println("Hello world");
 }
}
```

## # Java platform Components:-

- JDK:- (Java Development Kit):  
Contains compiler (javac), JRE,  
and tools to develop java applications.
- JRE:- (Java Runtime Environment):  
Contains JVM and libraries to  
run java apps.
- JVM:- (Java Virtual Machine): executes  
Java Bytecode, providing  
platform independence.

## # Applications of Java:-

- ① Web applications (eg. using Spring)
- ② Android apps (via android SDK)
- ③ Enterprise software (eg. ERP, banking)
- ④ Desktop applications (eg. Swing, JavaFX)
- ⑤ Scientific and embedded systems.



20/July/25

## Java Notes

1/1

Ques What is java programming language?

Ans Java is a high-level, object-oriented, and platform independent programming language. James Gosling at Sun Microsystems in 1995. It is widely used for building enterprise applications, mobile apps (Android), web servers and embedded systems.

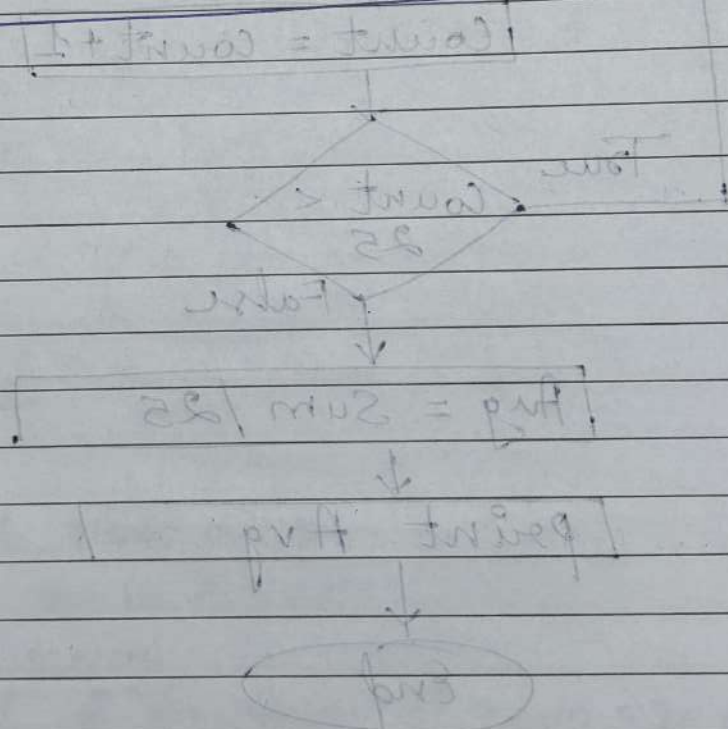
• Key characteristics of java:-

- ① Object-oriented:- Everything in java is based on objects and classes.
- ② Platform-independent:- Java code is compiled into bytecode which runs on the java virtual machine (JVM), making it portable across platforms.
- ③ Compiled and Interpreted:- Java source code is compiled to bytecode (.class files) which is interpreted by the JVM.
- ④ Strongly typed:- Variables must be declared with a type.



# pseudocode :-

- ① Start
- ② declare  $\text{count} = 0$ ,  $\text{sum} = 0$
- ③ while  $\text{count} < 25$
- ④     input score from user
- ⑤      $\text{sum} = \text{sum} + \text{score}$
- ⑥      $\text{count} = \text{count} + 1$
- ⑦ end while loop
- ⑧ Calculate average using  $\text{sum}/25$ .
- ⑨ Print Average
- ⑩ End.





\_/\_/\_

\* Non-Primitive Data types :- Such as String, array and classes.

# Type Conversion :-

Conversion happens when:

- a) type compatible
- b) destination type > Source type

⇒ byte → short → int → float → long → double

\* Data can be stored in the upcoming data type only

Eg:- byte → short  
byte → int

float → int (X) → not possible

\* This is also called widening conversion.

\* long a = 10; } long can be stored in double.  
double b = a;

destination type > Source type

\* Java does ~~not do~~ this conversion on its own, so it's also called implicit conversion.



## Data type

## Description

- ① byte → Stores whole numbers from -128 to 127
- ② short → Stores whole numbers from -32,768 to 32,767
- ③ int → Stores whole numbers from -2,147,483,648 to 2,147,483,647
- ④ long → Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
- ⑤ Float → Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits.
- ⑥ double → Stores fractional numbers. Sufficient for storing 15 to 16 decimal digits.
- ⑦ Boolean → Stores true or False values.
- ⑧ Char → Stores a single character/letter or ASCII values.



## # The main method :-

- The `main()` method is required and you will see it in every Java program.

→ `public static void main(String args[])`

- Any code inside the `main()` method will be executed.
- Just remember that every Java program has a class name which must match the filename, and that every program must contain the `main()` method.

Note:- The curly braces `{}` marks the beginning and the end of a block of code.

## # Java Data types :-

- Data types are divided into two groups:-

- ① Primitive Data types
- ② Non-primitive data types.

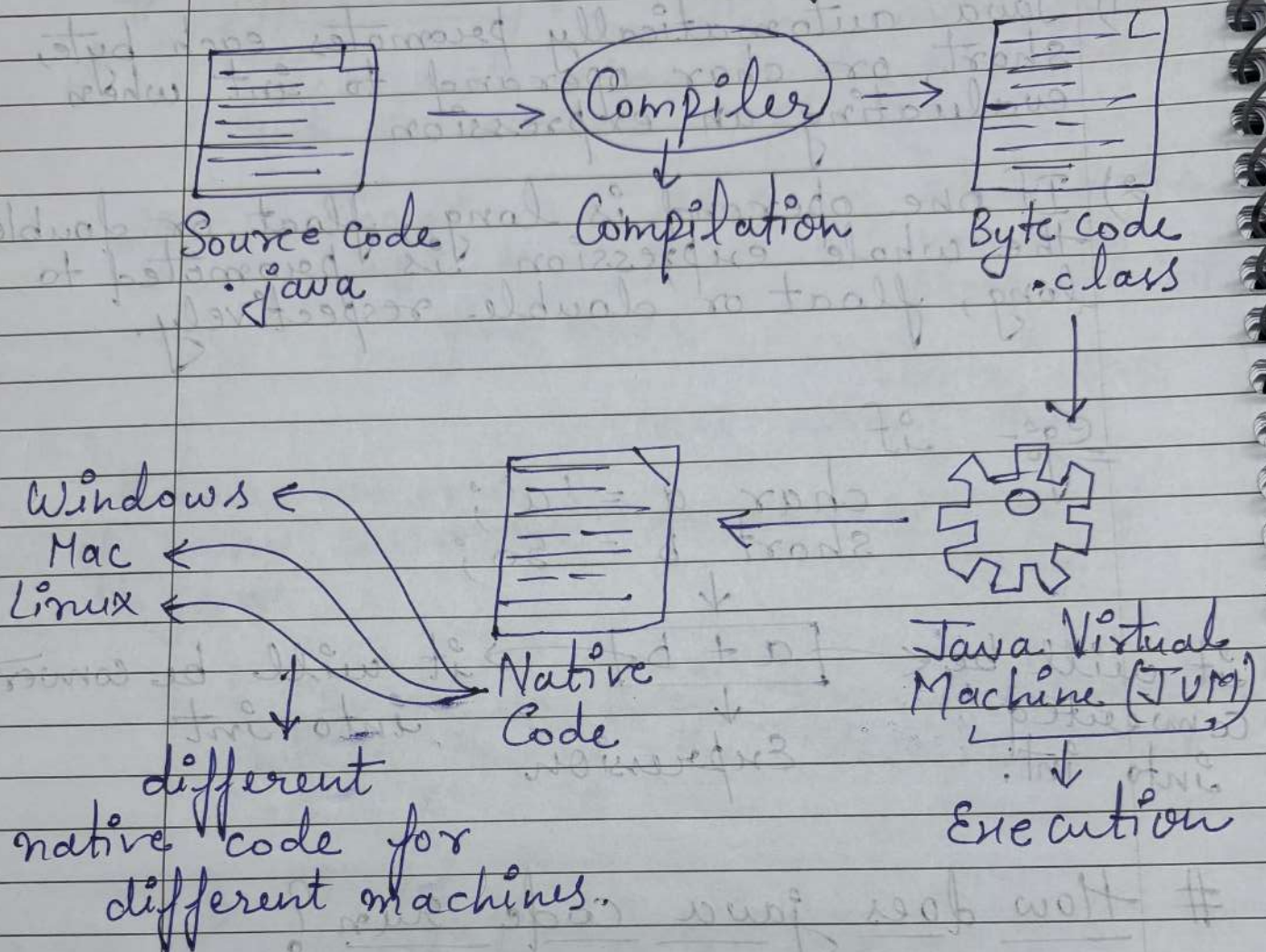
## # Primitive Data types :-

A primitive data type specifies the type of a variable and the kind of values it can hold.

→ There are eight primitive data types in Java:-



# How is our code running?



\* Java is a portable language.



## # Type Promotion in Expression :-

- 1) Java automatically promotes each byte, short, or char operand to int when evaluating an expression.
- 2) If one operand is long, float or double the whole expression is promoted to long, float or double respectively.

eg:- if

```
char a = 'a';
short b = 50;
```

↓

[a + b]

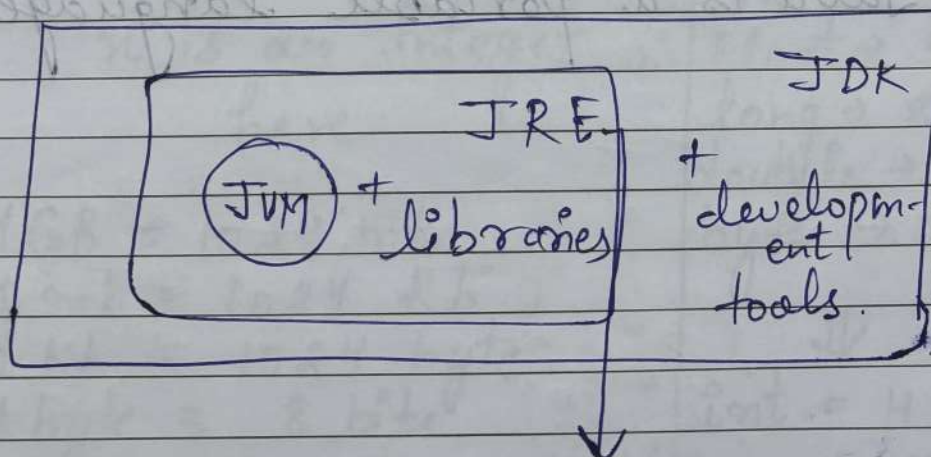
↓

Expression

It will be converted into int.

It will be converted into int

## # How does java code run?



Java Runtime Environment



# Type Casting :- Type Casting is a conversion which java does not allow to do, so we do such conversions explicitly.

eg:-

```
Float a = 25.0;
int b = a;
```

gives error

storing the value of float to int.

↓

does

Java does not allow that

- \* throws an error of lossy conversion in java.

Because of Loss of Data.

So to do such type of conversion while losing the data, we write this :-

⇒ Float a = 25.0;  
int b = (int) a;

does not give error.

\* Type Casting is also called narrowing conversion.

\* It is also called explicit conversion.